Lesson 13 Teacher's Key: A Hit That Reacts

TIC-80 - Instructor Reference Only

VIDEO GAME CREATION

What This Lesson Teaches

Extending each enemy's table entry with a new 'flash' field, and adding a genuinely new per-frame update loop to manage it - TIC-80's version of GameMaker's Alarm, built from the same plain Lua tools (tables, loops, variables) this engine has used all course, since it has no blocking wait either.

Level 1 - Full Worked Solution

- e.flash = 6 marks a hit enemy for delayed removal instead of removing it immediately - the enemy stays in the table, just in a new "flashing" state.
- The new update loop runs every frame from TIC(), counting down every flashing enemy's and every explosion's timer, removing each once it reaches 0 - this is the engine's substitute for a blocking wait.
- Score, enemies_remaining, and the win-check all still happen immediately in the hit branch, completely unchanged from Lesson 12 - only the actual table.remove() for the enemy is delayed.

Why This Structure

- Adding a flash field to each enemy's own table entry (rather than a separate parallel table) keeps each enemy's full state - position, alive-ness, now flash status - in one place, the same 'one entity, one table entry' discipline TIC-80 has used since Lesson 4.
- A dedicated explosions table (separate from enemies) is appropriate here because explosions and enemies are genuinely different kinds of things with different lifetimes and different drawing logic - unlike the flash field, which is naturally just an enemy's temporary state.

Level 2 / Level 3 Talking Points

- Ask students to trace what happens to a single enemy's table entry across several frames after being hit: e.flash=6, drawn as the flash sprite, decremented each frame, removed at 0 - a good comprehension check for how table-based state persists across frames.
- For students comparing engines: point out that TIC-80's flash-countdown-in-a-table and GameMaker's alarm[0]-plus-Alarm-event solve the identical problem - both engines lack a blocking wait, so both split 'trigger' and 'finish' into two separate places in code.

Common Student Mistakes

- Removing the enemy from the table immediately in the hit branch (old habit from Lesson 4-12), which defeats the whole point - the flash field approach requires NOT removing it there anymore.
- Forgetting to call the new update loop from TIC() every frame, leaving flashing enemies and explosions on screen forever.
- Drawing logic that doesn't check e.flash at all, so hit enemies never actually show the flash sprite even though the field is being set correctly.